3-Hour Lab Activity: File-Driven Job-Hunting Agent

Resume tailoring, interview preparation, application tracking, and reminders using job posters, resumes, and a slides-based knowledge base

Course Context	Agentic AI / AI Agents / Career Preparation
Duration	3 Hours
Level	Basic implementation; uniqueness and useful ideas carry additional marks
Final Deliverable	GitHub repository with working agent, structured input folders, generated outputs, and reflection

1. Activity Scenario

Students will build a beginner-level Job-Hunting Agent that reads files from clearly defined folders and helps them manage their job application workflow. The agent will not depend on hard-coded text only; instead, students will paste real or sample job posters, course PPT/PDF knowledge material, and resumes into folders. The agent will read these files, analyze them, generate outputs, and maintain an application tracker.

- Students place job posters or job descriptions in one folder.
- Students place course PPTs/PDFs or exported slide text in one folder to create the interview knowledge base.
- Students place resume files in one folder.
- The agent reads these files, extracts useful text, and performs basic analysis.
- The agent tailors resumes according to selected jobs.
- The agent generates interview questions from the knowledge base.
- The agent keeps track of application status, interview schedules, reminders, and next actions.

2. Learning Objectives

- Design a simple file-driven AI agent using the GAME framework.
- Organize project inputs into folders for job posters, resumes, and KB material.
- Implement basic file reading, keyword extraction, matching, and output generation in Python.
- Generate a tailored resume improvement report based on a job description.
- Generate interview questions using the course/job-preparation knowledge base.
- Maintain a simple job application tracker with status and reminder fields.
- Use GitHub for version control and final submission.
- Demonstrate originality by adding useful and unique features beyond the minimum requirements.

3. Agent Name and Purpose

Suggested agent name: CareerPrep Job-Hunting Agent

Suggested repository name: job-hunting-agent

Purpose: Help a student convert scattered job-search material into a structured workflow: job analysis, resume tailoring, interview preparation, application status tracking, and reminders.

4. Required Folder-Based Inputs

Folder	Student Must Place	Accepted Basic Format	Purpose
input_jobs/	Job posters, job descriptions, internship ads	TXT preferred; PDF optional if implemented	Agent reads job requirements and target skills
input_resumes/	Student resume/profile	TXT preferred; PDF/DOCX optional if implemented	Agent reads current profile and skills
input_kb/	Course slides, job fair slides, interview PDFs/PPT exports	TXT preferred; PDF optional if implemented	Agent generates interview questions from trusted material
outputs/	Generated reports	TXT files	Agent saves results
tracker/	Application tracker files	CSV / TXT	Agent tracks applications, statuses, interview dates, and reminders

5. Expected Repository Structure

```
job-hunting-agent/
|
|-- README.md
|-- app.py
|-- requirements.txt
|-- reflection.md
|
|-- input_jobs/
|   |-- job_poster_01.txt
|   |-- job_poster_02.txt
|
|-- input_resumes/
|   |-- my_resume.txt
|
|-- input_kb/
|   |-- job_fair_slides_notes.txt
|   |-- interview_preparation_notes.txt
|
|-- outputs/
|   |-- job_analysis_report.txt
|   |-- tailored_resume_suggestions.txt
|   |-- interview_questions.txt
|   |-- preparation_plan.txt
|   |-- final_agent_report.txt
|
|-- tracker/
|   |-- applications.csv
|   |-- reminders.txt
|
|-- samples/
    |-- sample_job_poster.txt
    |-- sample_resume.txt
    |-- sample_kb.txt
```

6. GAME Framework Design

Goal

Help a student search for jobs, analyze job posters, tailor a resume according to job requirements, generate interview preparation questions from course slides/KB, and maintain application status with reminders.

Actions

Action	Input	Output
Read Files	input_jobs, input_resumes, input_kb	Loaded text content
Analyze Job Poster	Job poster text	Skills, role, responsibilities, keywords
Analyze Resume	Resume text	Existing skills, projects, tools, strengths
Match Resume with Job	Job + resume	Match score, matched skills, missing skills
Tailor Resume	Job + resume + gaps	Resume improvement suggestions and bullet points
Search KB	JD skills + KB text	Relevant slide notes/topics
Generate Interview Questions	KB + JD skills	Technical, HR, project, and behavioral questions
Track Application	Company, role, status, date	applications.csv updated
Generate Reminders	Interview date / follow-up date	reminders.txt with pending actions

Memory

- Current selected job poster and extracted job skills.
- Current resume and extracted candidate skills.
- Knowledge-base notes from slides.

- Generated reports and interview questions.
- Application status history in tracker/applications.csv.
- Interview dates, follow-up dates, and reminders in tracker/reminders.txt.

Environment

- Local project folders and text files.
- GitHub repository for submission.
- Optional GitHub Copilot or ChatGPT for coding support.
- Optional PDF/DOCX extraction libraries for advanced students.

7. Three-Hour Lab Plan

Time	Activity	Outcome
0-15 min	Instructor briefing and scenario	Students understand the job-hunting agent workflow
15-30 min	GitHub repository and folder setup	Repository with required folders
30-55 min	Add job posters, resumes, and KB files	Input folders populated
55-80 min	Implement file reader and basic extraction	Agent reads all folders
80-110 min	Implement JD/resume matching and skill-gap report	Basic analysis output generated
110-135 min	Implement resume tailoring and interview question generation	Career-preparation outputs generated
135-160 min	Implement application tracker and reminders	Application status/reminder files updated
160-175 min	Testing, uniqueness improvements, README, reflection	Submission quality improved
175-180 min	Push to GitHub and submit	Repository link submitted

8. Minimum Functional Requirements

- The agent must read at least one job poster from input_jobs/.
- The agent must read at least one resume from input_resumes/.
- The agent must read at least one KB file from input_kb/.
- The agent must generate a job analysis report.
- The agent must generate a skill-gap report.
- The agent must generate resume tailoring suggestions.
- The agent must generate interview questions using KB content.
- The agent must maintain an application tracker with application status.
- The agent must generate reminder text if interview/follow-up date is provided.

9. Basic Workflow of the Agent

```
1. Read all job posters from input_jobs/
2. Read resume from input_resumes/
3. Read knowledge-base files from input_kb/
4. Extract keywords from job posters
5. Extract skills/projects from resume
6. Compare job requirements with resume
7. Generate skill-gap report
8. Generate tailored resume suggestions
9. Generate interview questions from KB and job skills
10. Ask/update application status
11. Save application tracker
12. Generate reminders for interviews/follow-ups
13. Save all outputs in outputs/ and tracker/
```

10. Application Status Tracker

The tracker is a mandatory part of the activity. Students must create and update tracker/applications.csv. This makes the agent more practical because it does not only prepare the student for one job; it also supports follow-up actions.

Field	Description	Example
application_id	Unique ID for each application	APP-001
company	Company name	ABC Tech
role	Job/internship title	Junior AI Engineer Intern
source	Where job was found	LinkedIn / Poster / WhatsApp / Rozee
status	Current status	Not Applied / Applied / Shortlisted / Interview Scheduled / Rejected / Offered
applied_date	Date application was submitted	2026-04-28
interview_date	Interview date if scheduled	2026-05-03
follow_up_date	Date to follow up	2026-05-06
next_action	What student must do next	Revise ML basics and prepare project explanation
notes	Additional remarks	Resume tailored for Python and ML role

```
application_id,company,role,source,status,applied_date,interview_date,follow_up_date,next_action,notes
APP-001,ABC Tech,Junior AI Engineer Intern,LinkedIn,Interview Scheduled,2026-04-28,2026-05-03,2026-05-06,Prepare Python and ML questions,Resume tailored and submitted
```

11. Reminder Logic

At a basic level, students may implement reminders as text output. Advanced students may compare dates with today's date and show urgent reminders.

- If status is Interview Scheduled, generate an interview preparation reminder.
- If follow-up date is available, generate a follow-up reminder.
- If status is Not Applied, remind the student to tailor resume and apply.
- If status is Applied and no response has been received, remind the student to follow up after a few days.

```
Example reminder output:
- APP-001: Interview scheduled with ABC Tech on 2026-05-03.
  Next action: Revise Python, ML basics, and explain your projects clearly.
- APP-002: Application submitted. Follow up on 2026-05-06 if no response is received.
```

12. Starter Code for app.py

This starter code is intentionally simple. Students may improve it using better extraction, PDF reading, menu systems, Streamlit, JSON, or LLM API integration.

```
import os
import csv
from datetime import datetime

JOB_DIR = "input_jobs"
RESUME_DIR = "input_resumes"
KB_DIR = "input_kb"
OUTPUT_DIR = "outputs"
TRACKER_DIR = "tracker"

KEYWORDS = [
    "python", "machine learning", "data preprocessing", "github", "git",
    "api", "prompt engineering", "sql", "communication", "problem solving",
    "oop", "database", "jupyter", "pandas", "numpy", "deep learning",
    "html", "css", "flask", "streamlit", "resume", "interview"
]


def ensure_folders():
    for folder in [JOB_DIR, RESUME_DIR, KB_DIR, OUTPUT_DIR, TRACKER_DIR]:
        os.makedirs(folder, exist_ok=True)


def read_text_files(folder):
    combined_text = ""
    file_count = 0
```

```
    for filename in os.listdir(folder):
        if filename.lower().endswith(".txt"):
            path = os.path.join(folder, filename)
            with open(path, "r", encoding="utf-8") as file:
                combined_text += f"\\n\\n--- FILE: {filename} ---\\n"
                combined_text += file.read()
                file_count += 1

    return combined_text, file_count


def save_text(path, content):
    with open(path, "w", encoding="utf-8") as file:
        file.write(content)


def extract_keywords(text):
    text_lower = text.lower()
    found = []
    for keyword in KEYWORDS:
        if keyword in text_lower:
            found.append(keyword)
    return found


def compare_skills(job_skills, resume_skills):
    matched = [skill for skill in job_skills if skill in resume_skills]
    missing = [skill for skill in job_skills if skill not in resume_skills]
    score = 0 if not job_skills else round((len(matched) / len(job_skills)) * 100, 2)
    return matched, missing, score


def generate_job_analysis(job_text, job_skills):
    report = "Job Analysis Report\\n==================\\n\\n"
    report += "Skills/keywords found in job posters:\\n"
    for skill in job_skills:
        report += f"- {skill}\\n"
    return report


def generate_skill_gap_report(job_skills, resume_skills, matched, missing, score):
    report = "Skill Gap Report\\n================\\n\\n"
    report += f"Match Score: {score}%\\n\\n"
    report += "Matched Skills:\\n"
    for skill in matched:
        report += f"- {skill}\\n"
    report += "\\nMissing Skills:\\n"
    for skill in missing:
        report += f"- {skill}\\n"
    return report


def generate_resume_suggestions(job_skills, missing):
    output = "Tailored Resume Suggestions\\n==========================\\n\\n"
    output += "Suggested improvements according to job posters:\\n"
    for skill in job_skills:
        output += f"- Add or improve resume evidence related to {skill}.\\n"

    output += "\\nSuggested resume bullets:\\n"
    output += "- Developed Python-based academic projects with clear documentation.\\n"
    output += "- Used GitHub for project version control and README-based documentation.\\n"
    output += "- Applied problem-solving skills to complete course and project tasks.\\n"

    if missing:
```

```
        output += "\\nSkills to improve before applying/interview:\\n"
        for skill in missing:
            output += f"- {skill}\\n"

    return output


def generate_interview_questions(job_skills, kb_text):
    questions = "Interview Questions\\n==================\\n\\n"
    questions += "Technical questions based on job posters:\\n"
    for skill in job_skills:
        questions += f"- Explain your understanding of {skill}.\\n"
        questions += f"- How have you used {skill} in a project or course activity?\\n"

    questions += "\\nHR and behavioral questions:\\n"
    questions += "- Tell me about yourself.\\n"
    questions += "- Why are you interested in this role?\\n"
    questions += "- Describe your best academic/project work.\\n"
    questions += "- What are your strengths and weaknesses?\\n"
    questions += "- Why should we select you?\\n"

    questions += "\\nQuestions inspired by KB/slides:\\n"
    kb_lines = [line.strip("- ").strip() for line in kb_text.splitlines() if line.strip()]
    for line in kb_lines[:10]:
        questions += f"- How would you explain this point in an interview: {line}?\\n"

    return questions


def create_or_update_tracker():
    path = os.path.join(TRACKER_DIR, "applications.csv")
    if not os.path.exists(path):
        with open(path, "w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow([
                "application_id", "company", "role", "source", "status",
                "applied_date", "interview_date", "follow_up_date", "next_action", "notes"
            ])
            writer.writerow([
                "APP-001", "Sample Company", "AI Intern", "Job Poster",
                "Not Applied", "", "", "", "Tailor resume and apply", "Sample row"
            ])
    return path


def generate_reminders():
    tracker_path = os.path.join(TRACKER_DIR, "applications.csv")
    reminders = "Application Reminders\\n====================\\n\\n"

    if not os.path.exists(tracker_path):
        return "No tracker file found."

    with open(tracker_path, "r", encoding="utf-8") as file:
        reader = csv.DictReader(file)
        for row in reader:
            app_id = row.get("application_id", "")
            company = row.get("company", "")
            role = row.get("role", "")
            status = row.get("status", "")
            interview_date = row.get("interview_date", "")
            follow_up_date = row.get("follow_up_date", "")
            next_action = row.get("next_action", "")

            if status.lower() == "interview scheduled":
                reminders += f"- {app_id}: Interview scheduled for {role} at {company} on {interview_date}. Next
```

```
action: {next_action}\\n"
            elif status.lower() == "not applied":
                reminders += f"- {app_id}: Not applied yet for {role} at {company}. Tailor resume and apply.\\n"
            elif status.lower() == "applied":
                reminders += f"- {app_id}: Application submitted to {company}. Follow up on {follow_up_date}.\\n"

    return reminders


def run_agent():
    ensure_folders()

    job_text, job_count = read_text_files(JOB_DIR)
    resume_text, resume_count = read_text_files(RESUME_DIR)
    kb_text, kb_count = read_text_files(KB_DIR)

    if job_count == 0 or resume_count == 0 or kb_count == 0:
        print("Please add .txt files in input_jobs, input_resumes, and input_kb folders.")
        return

    job_skills = extract_keywords(job_text)
    resume_skills = extract_keywords(resume_text)
    matched, missing, score = compare_skills(job_skills, resume_skills)

    job_report = generate_job_analysis(job_text, job_skills)
    gap_report = generate_skill_gap_report(job_skills, resume_skills, matched, missing, score)
    resume_suggestions = generate_resume_suggestions(job_skills, missing)
    interview_questions = generate_interview_questions(job_skills, kb_text)

    create_or_update_tracker()
    reminders = generate_reminders()

    final_report = "CareerPrep Job-Hunting Agent Report\\n"
    final_report += f"Generated on: {datetime.now()}\\n"
    final_report += "==================================\\n\\n"
    final_report += job_report + "\\n"
    final_report += gap_report + "\\n"
    final_report += resume_suggestions + "\\n"
    final_report += interview_questions + "\\n"
    final_report += reminders + "\\n"

    save_text(os.path.join(OUTPUT_DIR, "job_analysis_report.txt"), job_report)
    save_text(os.path.join(OUTPUT_DIR, "skill_gap_report.txt"), gap_report)
    save_text(os.path.join(OUTPUT_DIR, "tailored_resume_suggestions.txt"), resume_suggestions)
    save_text(os.path.join(OUTPUT_DIR, "interview_questions.txt"), interview_questions)
    save_text(os.path.join(OUTPUT_DIR, "final_agent_report.txt"), final_report)
    save_text(os.path.join(TRACKER_DIR, "reminders.txt"), reminders)

    print("Agent completed successfully.")
    print(f"Job files read: {job_count}")
    print(f"Resume files read: {resume_count}")
    print(f"KB files read: {kb_count}")
    print(f"Match score: {score}%")
    print("Outputs saved in outputs/ and tracker/ folders.")


if __name__ == "__main__":
    run_agent()
```

13. Student Input Preparation

- Job posters may be copied from LinkedIn, Rozee, company websites, WhatsApp posters, or internship announcements. Students must remove private contact information if not needed.

- Course PPTs/PDFs should be converted to text for the basic version. Students may copy important slide content into input_kb/*.txt.
- Resume should be pasted as text in input_resumes/my_resume.txt. Students may use a sample resume if they do not want to use personal data.
- Each folder must contain at least one file before running the agent.

14. Output Requirements

- outputs/job_analysis_report.txt
- outputs/skill_gap_report.txt
- outputs/tailored_resume_suggestions.txt
- outputs/interview_questions.txt
- outputs/final_agent_report.txt
- tracker/applications.csv
- tracker/reminders.txt

15. Unique Ideas That Can Carry Extra Marks

- Menu-based interface to select a particular job poster and resume.
- PDF reading for job posters, resumes, or KB material.
- DOCX resume reading.
- Streamlit web interface.
- Application dashboard showing counts by status.
- Reminder urgency: today, tomorrow, this week, overdue.
- Resume quality score.
- Cover-letter generator.
- LinkedIn message generator for recruiter outreach.
- Company-wise preparation checklist.
- Interview answer hints based on KB.
- Project-to-JD mapping: which student project supports which job requirement.
- Export final report as PDF.
- JSON-based memory file.
- Proper LLM API integration with safe prompt design.

16. Marking Rubric

Component	Marks
GitHub repository and required folder structure	5
Job poster folder and file reading	5
Resume folder and file reading	5
KB folder and file reading	5
Job/resume matching and skill-gap report	8
Resume tailoring suggestions	6
Interview questions from KB	6
Application tracker with status fields	6
Reminder generation for interview/follow-up	5
README, reflection, and clear documentation	4
Uniqueness, creativity, and useful extra ideas	10
Total	65

17. Final Submission Form

```
Job-Hunting Agent Submission Form

1. Student Name:
```

```
2. Roll Number:
3. Section:
4. GitHub Username:
5. GitHub Repository Link:

6. Number of job posters added in input_jobs/:
7. Number of resume files added in input_resumes/:
8. Number of KB files added in input_kb/:

9. Agent features implemented:
   - File reading from folders: Yes / No
   - JD analysis: Yes / No
   - Resume analysis: Yes / No
   - Skill-gap report: Yes / No
   - Resume tailoring: Yes / No
   - Interview questions from KB: Yes / No
   - Application tracker: Yes / No
   - Reminders: Yes / No

10. Current application statuses used in tracker:

11. Unique feature added by student:

12. LLM/Copilot usage:

13. Testing evidence:

14. Declaration:
I confirm that I built and tested this agent myself and understand the submitted code.

Student Signature:
Date:
```

18. Suggested Student Announcement

Dear Students,

In the upcoming 3-hour lab, you will build a file-driven Job-Hunting Agent. You are required to create separate folders for job posters, resumes, and course PPT/PDF knowledge material. The agent must read these files and generate resume tailoring suggestions, interview questions, application status tracking, and reminders for interviews or follow-ups.

This is a basic-level agent activity; however, uniqueness, creativity, and practical usefulness will carry marks. You may add features such as PDF reading, dashboard view, cover-letter generation, reminder urgency, or a Streamlit interface.

Please bring your laptop, GitHub account, sample resume, job posters, and course/job-preparation slides or notes.

19. Final Deliverable Checklist

- GitHub repository link submitted.
- input_jobs/ contains at least one job poster or JD file.
- input_resumes/ contains at least one resume file.
- input_kb/ contains at least one KB file from course slides/PDFs.
- Agent reads files from folders, not only hard-coded text.
- outputs/ contains generated reports.
- tracker/applications.csv contains application status records.
- tracker/reminders.txt contains interview/follow-up reminders.
- README.md explains how to run the agent.
- reflection.md explains what was built, tested, and improved.